

manache

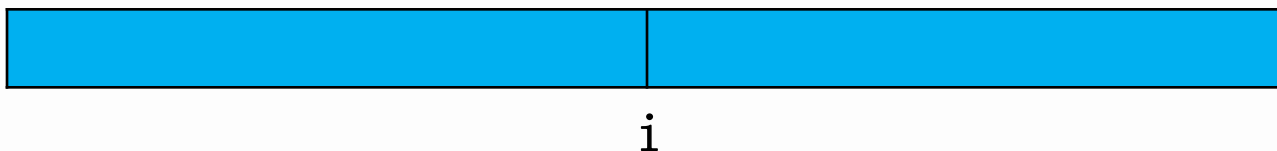
在前面学习的求最长回文子串的练习题中，我们学会了用二分+hash来求解，实际上有人发明了一个求解最长回文子串的更高效的办法——manache算法。并不是说这个算法和马拉车有点类似，只是单纯的因为他的发明者是manache而已。

manache算法的原理很简单，和kmp思想类似，都是不重复枚举，如果可以用到前面的答案，就不需要再从头开始枚举了。

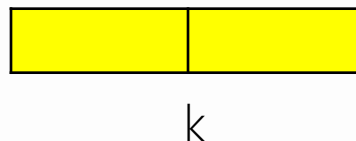
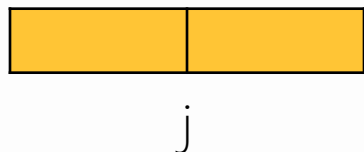
我们先思考一下，回文子串的暴力，当然是从每一个字符开始往左右两边延伸比较两个字符串是否相等，但是如果画一个图，就会发现里面有些重复了。

manache

以 i 为回文中心的回文串长度为 l_i



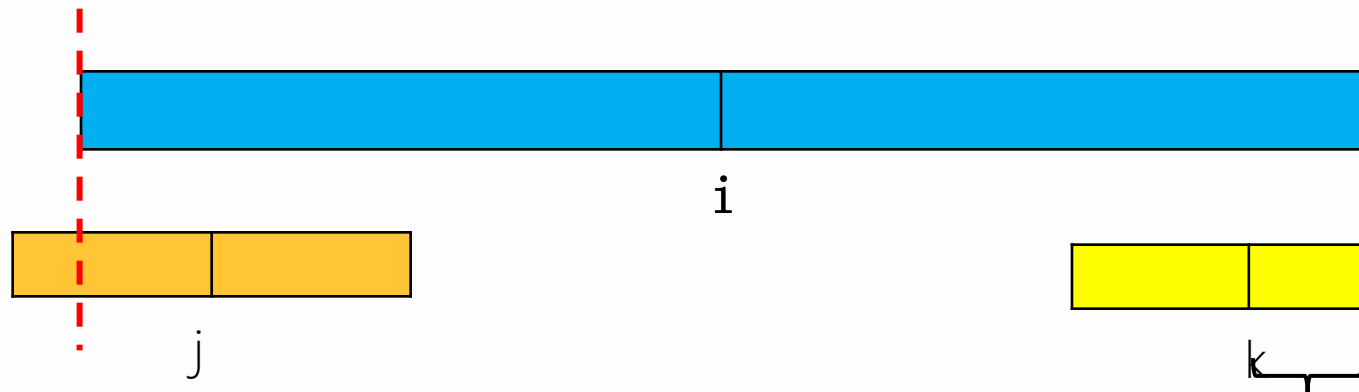
以 j 为回文中心的回文串长度为 l_j , 让 $j < l_i$, 此时有三种情况。



第一种情况：

如果以 j 为回文中心的回文串包含于 i 的回文串左边的时候，根据 i 的回文串的原理，那么很明显， i 的右边一定存在一个与 j 对称的位置 k ，他的回文串长度和 j 相等，所以 k 的回文长度我们就直接知道了。

manache

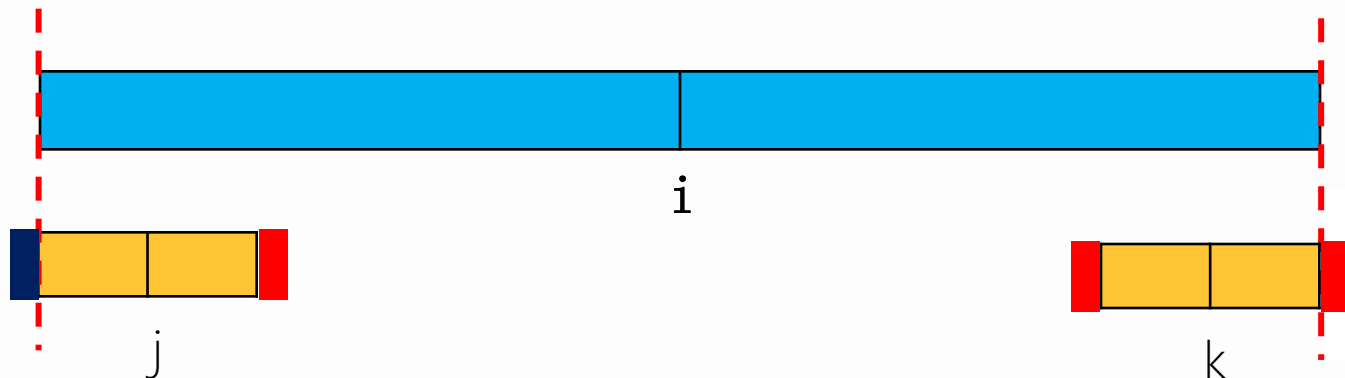


第二种情况：

如果以 j 为回文中心的回文串左边超出了 i 的回文串的范围，那么根据 i 的回文的原理，右边也有一个与 j 对称的点 k ，但是由于 j 有一部分超出了 i 的回文范围，那么 k 的右边也超出了，那我们能不能确定 k 的回文长度呢？完全是可以的！！后面未知的一部分一定和 j 的左边超出部分肯不回文，为什么呢？向下 i 的回文长度的意义，如果还回文，那么 i 的长度就不是 $|i|$ 了

$\text{Len}[k] = i - j$ (大概长度)

manache



第三种情况：

如果 j 的回文串的左边和 i 的回文串的左边刚好相等，那么此时是可以继续延伸的，如上图，就算 k 继续往两边延伸也不会和以前的 i ， j 的答案矛盾，所以这里一定要注意。

manache

对于超过 i 的回文范围的数据，我们就没有办法用到回文的性质了，所以只能老老实实的去计算，实际上我们发现，对于回文串长度内的位置都是直接得到答案，并没有重复计算，所以时间复杂度为 $O(n)$ 。

还有一个问题，因为存在双回文和单回文，很麻烦，我们直接都把他变成单回文(回文串长度为单数，回文中心为一个字符)，怎么操作呢？很简单，在每个空位置添加一个#(其他符号也可以)。比如

abcddba, 转换之后变成

#a#b#c#d#d#b#a#

原来字符串长度为 k ，新字符串长度为 $2*k+1$ ，一定是奇数，#就相当于以前的空位置回文。

manache

但是这样做最后要怎么计算回文长度呢？

a b c d d c b

010101016101010

#a#b#c#d#d#c#b#，我们标记一下回文半径

121212127212121

上下对比一下，有什么关系？

我们会发现对于任意个字符串，如果按照上面讲述的办法添加其他字符，那么对于新字符串得到的回文半径=原来字符串的回文半径+1，所以关系就出来了，我们只需要算出新字符串的回文半径，就可以得到原来字符串的回文半径，特别方便。

manache

但是这样做会有时会出现一点问题，比如aba
添加字符之后变成#a#b#a#，

那么b的回文半径为4，但是b是第3个字符串，那么在枚举回文长度的时候就会访问到-1这个元素，就会出现
问题，所以字符串一定要从1开始，同时可能要比较0字
符串和其中某个字符串是否相等，所以下标为0的位置的
字符不能和#以及字符串中出现的任一个字符相等，所以
我们可以让s[0]='\$'。

算法实现

第一步：变换字符串

```
int l=strlen(str);
s[0]='$';
s[1]='#';
int j=1;
for(int i=0;i<l;i++)
{
    s[++j]=str[i];
    s[++j]='#';
} //此时字符串为0--j, 有效范围为1--j
```


算法实现

第一步：manache算法

```
void manache(int l)
{
    len[0]=0;
    int mx=0; // 前面所有回文串能延伸的最远距离
    int id=0; // 最远距离对应的回文中心
    for(int i=1;i<=l;i++) // 枚举字符串
    {
        if(i<mx) // 如果i在回文范围之内
            len[i]=min(mx-i, len[id*2-i]); // 两者取最小
        else // 如果不在回文范围内
            len[i]=1; // 只有本身回文
        while(s[i-len[i]]==s[i+len[i]]) // 继续比较
            len[i]++;
        if(len[i]+i>mx) // 更新
        {
            mx=len[i]+i;
            id=i;
            sum=max(sum, len[i]);
        }
    }
}
```



练习题

洛谷	P3805	manache算法
洛谷	P4555	最长双回文串
洛谷	P1659	拉拉队排练
P0J	3974	Palindrome
BZOJ	3790	神奇的项链
BZOJ	3676	回文串



拓展 K M P

有两个字符串 a, b ，要求输出 b 与 a 的每一个后缀的公共前缀

输入：

aaaabaa

aaaaa

输出：

4 3 2 1 0 2 1

$\text{lena}, \text{lenb} \leq 100000$

拓展 K M P

题目要求第一个字符串的后缀与第二个字符串本身的公共前缀的长度。

aaaabaa的后缀分别为：

aaaabaa, aaabaa, aabaa, abaa, baa, aa, a

在与aaaaa求最长公共前缀

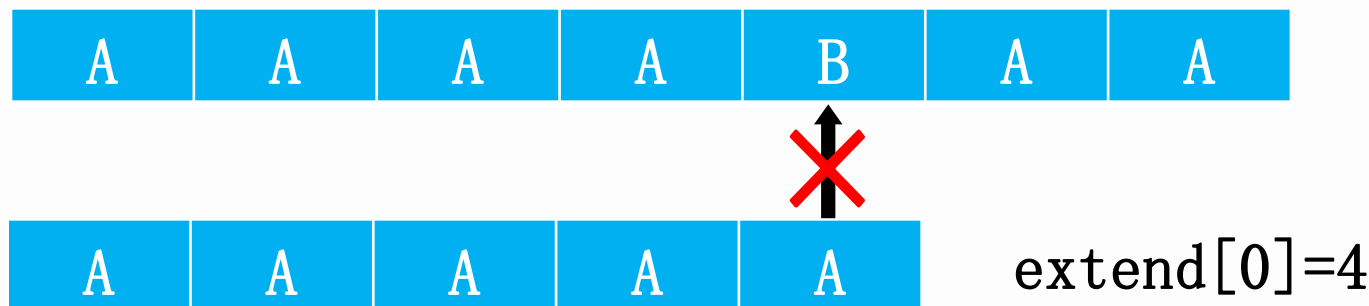
得到答案为4 3 2 1 0 2 1

暴力是很好处理的，但是要超时，主要是看能不能优化，既然是拓展kmp，那么很明显就是如果我们求出了 $\text{extend}[i]$ （第 i 个后缀和目标字符串的最长前缀），那么我们就可以很快的推出 $\text{extend}[i+1]$ 。

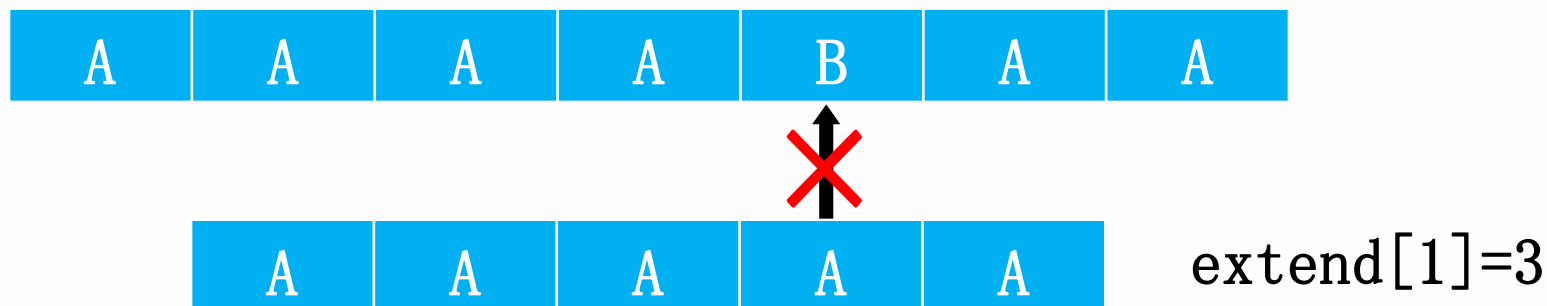
我们画图来看一下有没有规律

拓展 K M P

第一次匹配：

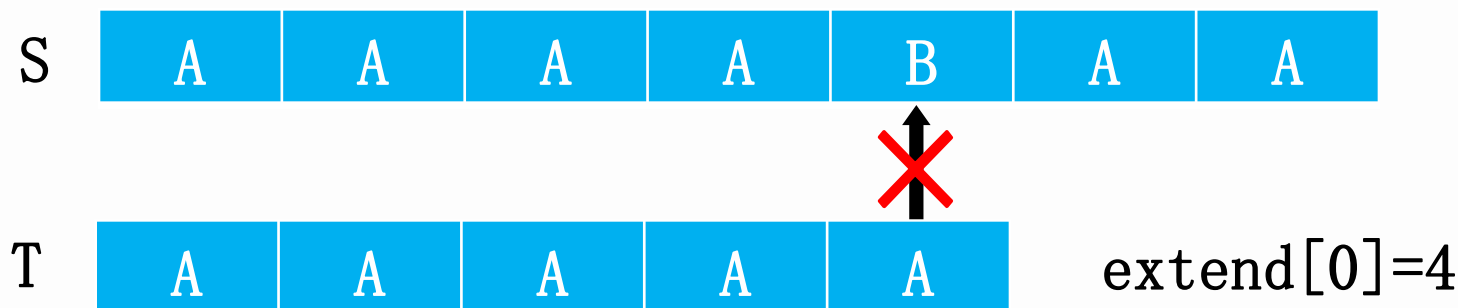


第二次匹配：



前面实际上可以匹配三个，但是我们只能从头开始比较，所以我们需要优化一下，去除重复。

拓展 K M P



因为 $\text{extend}[0]=4$, 那么:

$S[0\cdots 3]=T[0\cdots 3]$, $S[1\cdots 3]=T[1\cdots 3]$;

而我们接下来需要求解的是 $S[1\cdots X]$ 与 $T[0\cdots Y]$ 的公共前缀, 前面的 $S[1..3]$ 我们已经计算过一次了, 所以最好能不再比较了, 所以我们只需要判断出 $T[1..3]$ 的前缀和 $T[0..Y]$ 相比有多少是相同的就可以了。相同部分我们就不需要比较了。

所以, 总的来说, 我们第一步需要先求出来对于T的每一个后缀和T的最长公共前缀。

拓展 KMP

看上去和KMP有点类似，但是还是有区别，kmp求的是最长公共前后缀，而这里求得是T的每一个后缀和T的最长公共前缀。所以求法不一样，那该如何求解呢？

我们先假设 $1 \cdots i$ 的最远匹配位置为 p ，对应的后缀点为 x ，此时我们已经已经得到 $1 \cdots i$ 的next值，需要求 $i+1$ 的next值



$$T[x \cdots p] = T[0 \cdots p-x]$$



$$T[i+1 \cdots p] = T[i+1-x \cdots p-x]$$

设 $\text{next}[i+1-x] = \text{len}$. 并且 $i+1-x < i$ ，所以很明显 len 是已知的。
那么就有 $T[i+1-x \dots i+1-x+\text{len}-1] = T[0 \cdots \text{len}-1]$.

拓展 K M P

		x		i	i+1			p	
--	--	---	--	---	-----	--	--	---	--

0				i+1-x			p-x		
---	--	--	--	-------	--	--	-----	--	--

由前面我们已经知道 $T[i+1..p] = T[i+1-x..p-x]$

并且 $T[i+1-x..i+1-x+len-1] = T[0..len-1]$

所以此时分两组情况讨论。

第一种 $i+1+len \leq p$

S	0	...	Po	...	k	k+1	...	k+len	...	P	...
---	---	-----	----	-----	---	-----	-----	-------	-----	---	-----

T	0	...	Len-1	len	...	m-1
---	---	-----	-------	-----	-----	-----

那么很明显：

$T[i+len+1] \neq T[len]$ ，那么

$next[i+1] = len$

拓展 K M P

第二种情况：

$i+1+len > p$, 那么由于后面的位置重来没有被匹配过，所以我们并不知道，所以在判断就好。

S	0	...	P_0	...	k	k+1	...	P	P+1	...	...

T	0	...	...	$P-k$	...	$len-1$	...

也就是从 $T[p+1]$ 和 $T[p-k+1]$ 开始往后面匹配，知直到不可以匹配为止，并更新 `next` 的值。如果后面可以匹配，一定不要忘记更新 P 和 k (当前可以匹配的最远的距离与对应的后缀位置。)

拓展 K M P

next数组求好之后，最终要求的extend数组就好求了。

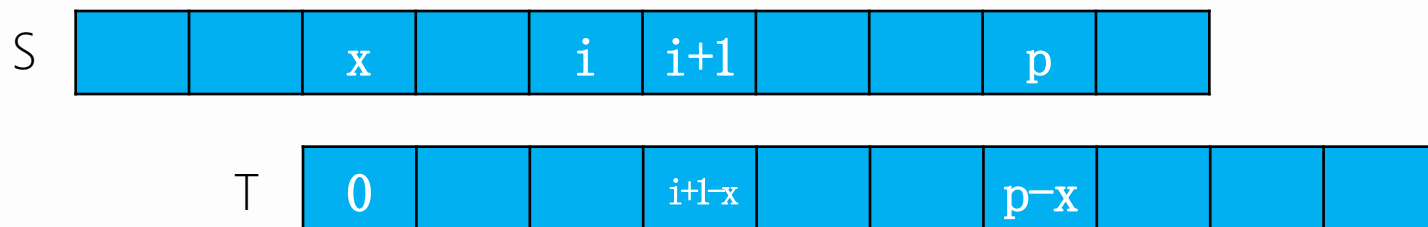
next数组：T的所有后缀和T的最长公共前缀

extend数组：S的所有后缀和T的最长公共前缀

所以实际上很明显了，和kmp的求next数组和最后的匹配一样，求extend和求next数组的求法一样。

拓展 K M P

假设对于S串来说，此时已经求出了 $\text{extend}[i]$ 的值，并且从 $1 \cdots i$ 匹配的最远的位置是 p ，对应的后缀的位置是 x 。



那么就有 $S[x \cdots p] = T[0 \cdots p-x]$. 其中的一部分
 $S[i+1 \cdots p] = T[i+1-x \cdots p-x]$.

又由我们已知的 next 数组可以知道。首先， $\text{next}[i+1-x]$ 的最远匹配位置不会超过 p ，那么假设 $\text{next}[i+1-x]$ 的长度为 len ，所以就有 $T[i+1-x \cdots i-x+\text{len}] = T[0 \cdots \text{len}-1]$.

后面也是一样，两种情况，即 $i+1+\text{len}$ 和 p 的位置关系。

拓展 K M P

```
void getNext()//求s2的next数组
//这里的next和kmp的next不一样，这里匹配的是s2[i,l2-1]和s2
{
    nextn[0]=l2;
    int i=0;
    while(s2[i]==s2[i+1]&& i+1<l2)
        i++;
    nextn[1]=i;//一定要先求出nextn[1]的值
    int po=1;//此时最远的距离是next[i]的值，不能包括next[0]
    for(i=2;i<l2;i++)
    {
        if(nextn[i-po]+i<nextn[po]+po)//ppt上的证明
            nextn[i]=nextn[i-po];
        else
        {
            int j=nextn[po]+po-i;//j表示当前可以匹配长度
            if(j<0) j=0;//如果i>p
            while(i+j<l2&& s2[j]==s2[i+j])
                j++;
            nextn[i]=j;
            po=i;//更新po的值
        }
    }
}
```

拓展 K M P

```
void excmp()
{
    getnext(); // 先求next数组
    int i=0;
    while(s1[i]==s2[i]&& i<l1&& i<l2)
        i++;
    ex[0]=i; // 先预处理出来ex[0]的值
    int po=0;
    for(int i=1; i<l1; i++)
    {
        if(nextn[i-po]+i<ex[po]+po)
            ex[i]=nextn[i-po];
        else
        {
            int j=ex[po]+po-i; // j表示下一个应该匹配的长度
            if(j<0) j=0;
            while(i+j<l1&& j<l2&& s1[i+j]==s2[j])
                j++;
            ex[i]=j;
            po=i;
        }
    }
}
```



练习题

洛谷 5410 拓展KMP

POJ 2752 荒岛的龟

POJ 3080 Blue Jeans

POJ 2752

POJ 3461

BZOJ 3670



后缀排序

读入一个字符串，请把这个字符串的所有非空后缀按字典序从小到大排序，然后按顺序输出后缀的第一个字符在原串中的位置。位置编号为1到n。

输入：
ababa

输出：
5 3 1 4 2

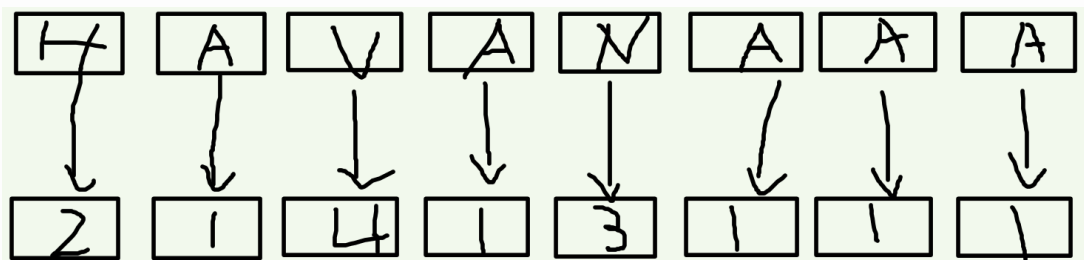
$N \leq 10^6$

后缀排序

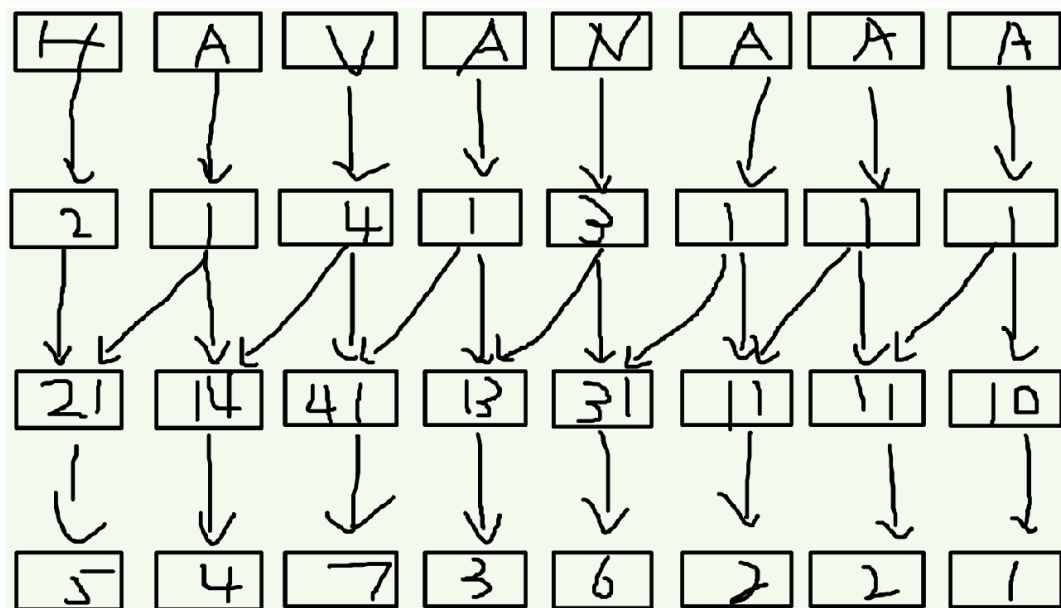
这是一道后缀数组的简单模板题。首先，我们思考暴力做法，相当于对 n 个字符串进行排序，排序时间复杂度为 $O(n \log n)$ ，每次比较两个字符串时间最坏复杂度为 $O(n)$ ，所以综的时间复杂度为 $n^2 \log n$ ，因为每一位字符按照字典序排序，所以还没有办法进行hash。

所以，需要用到一种新的字符串算法——后缀数组(SA)，后缀数组代码比较短，但是辅助数组较多，不太容易理解，所以一定要弄明白他的具体原理

后缀数组



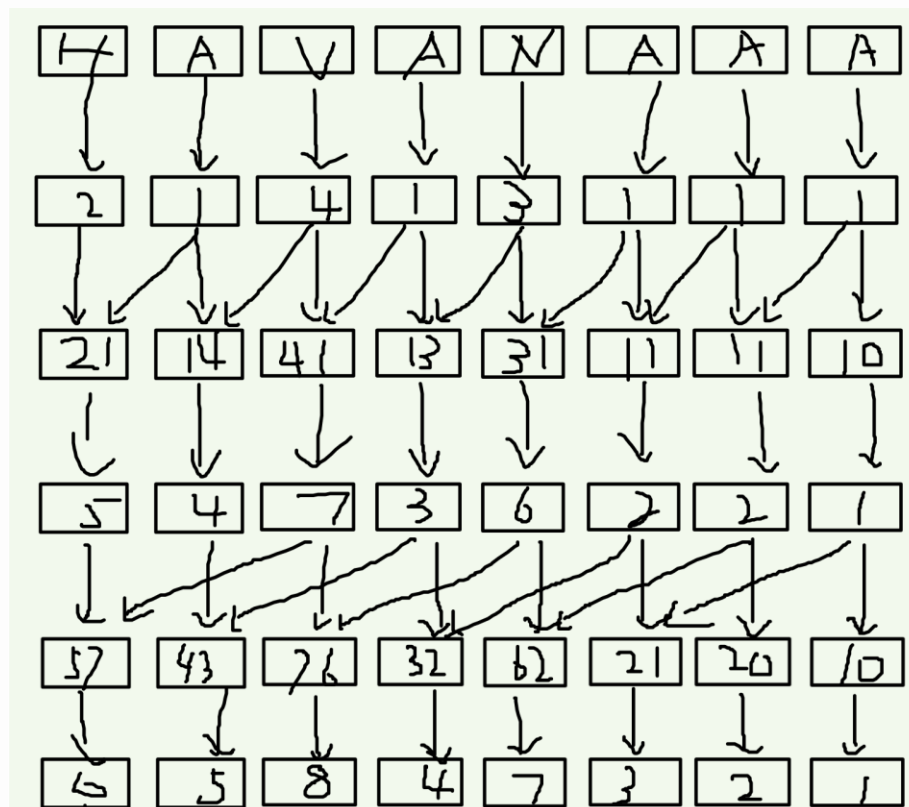
以i开头长度为1的字符串的排名



以i开头长度为2的字符串的排名(看清楚在长度为1的字符串上的变化)

后缀数组

我们会发现这样做时间效率并没有任何变化，那么对于这种依次枚举，我们要怎么才能让他的效率更高呢？没错，倍增！关键是如何实现倍增呢，也就是直接合并两块而不是两个。



也就是说HA排名为5，AV排名为4，VA排名为7，AN排名为3，那么合并HA和VA之和排名就是(5, 7)，这里我们理解为HA的排名为第一关键字，VA的排名为第二关键字，先比较第一关键字，如果有大小关系直接出结果，如果相等，再比较第二关键字，不影响原来结果，这样就可以把一个 n 优化为 $\log n$ 。

后缀数组

这样总的时间复杂度为 $n \log n \log n$ ，还是过不了100w，那还可以如何优化呢？排序这里可以优化，在这种情况下，我们可以考虑基数排序，基数排序实际上就是桶排序的优化，这里不是有两个关键字吗，我们可以按照两个关键字来排序。简单来说，就是我们先给定 n 个大桶有序放好，再再每个桶中有序放入 m 个小桶。

为什么这样做可以呢？比如在一个字符串中 a 出现了5次， b 出现了3次， c 出现了四次，那么以 b 开头的后缀的排名的范围我们可以知道吗？很明显是6—8，因为只要他的首位是 b ，那么后面他的字符串无论怎么变化，他的排名的范围就固定了，同样，无论以其他字符开头的字符串后面是什么，都不会影响到以 b 开头的字符串的排名。

后缀数组

那这样就很好解决了，我们先定义四个数组

$sa[i]$: 第一关键字排名为 i 的后缀的首字符的位置

$rak[i]$: 第 i 个位置开始的后缀的排名

$tp[i]$: 每个字符串都是有倍增组成的，所以实际上他是两部分，前面是第一个关键字，后面是第二关键字， tp 表示第二关键字排名为 i 的首字符位置

$tax[i]$: 记录长度为 w ，排名为 i 的当前后缀的个数。比如 $abab$ ，当长度为 2 时，排名为 1 的后缀有两个，即两个 ab 。在计算过程中，字符串长度不足，默认添加空字符，不影响答案。

我们先来学习含有两个关键字的基数排序，时间复杂度为 $O(n)$ 。

基数排序

```
void jsort()
{
    for(int i=0;i<=m;i++)    tax[i]=0;//初始化
    for(int i=1;i<=l;i++)    tax[rak[i]]++;//统计每个相同单词出现的数量
    for(int i=1;i<=m;i++)    tax[i]+=tax[i-1];//前缀和
    //前缀和可以直接统计比当前名次小的后缀有多少个
    for(int i=l;i>=1;i--)//从大大小枚举
        sa[tax[rak[tp[i]]]--]=tp[i];
    //我先找到第二关键字最大的位置tp[l]
    //对于第二关键字最大的，如果第一关键字相同，那么他就是在此基础上最大的
    //所以排名就为tax[],tax前缀和的用途就出来了，然后再--，表示该位置被占用了
    //那么sa[]就可以用tp[i]表示了
}
```

```
m=75;//统计一个所有字符的总数量, 'z'-'0'
for(int i=1;i<=l;i++)
{
    rak[i]=s[i]-'0'+1;//因为包含大小写和数字，字符变数字
    //当只有一个字符的时候，字符大小就是它的排名
    tp[i]=i;//第二关键字就是位置
}
jsort();//基数排序
```

后缀数组

```
for(int i=1;i>=1;i--)//从大大小枚举  
sa[tax[rak[tp[i]]--]]=tp[i];
```

前面三句代码就很好理解，求解排名前缀和 $\text{tax}[i]$ ，最后一句话，首先，是按照 $1 \sim 1$ 来枚举，字符串长度为 1 ，那么后缀的排名就是 $1 \sim 1$ ， 1 表示排名最后的那一个，那么 $\text{tp}[1]$ 就表示第二关键字排名最大的位置，那么 $\text{rank}[\text{tp}[i]]$ 就表示该位置的排名，前面我们已经说了，因为有两个关键字，那么当第一个关键字确定之后，第二个关键字不同的字符串他们的排名关系一定是连续的，并且不受其他字符串的影响，又由于该字符串的第二关键字是最大的，那么不就是 $\text{rank}[]$ 的排名吗(最后一个)，即 $\text{tax}[\text{rak}[\text{tp}[i]]]$ 。然后一就很好理解了，后面的再出现的排名在他之前，就 tax 值--。既然该位置的排名确定了，那么 sa 的值也就确定了，位置就是 $\text{tp}[i]$ 的位置。

后缀数组

```
p=0;
for(int i=1;i<=w;i++)
tp[++p]=1-w+i; //对于前面w个，相没有第二关键字，表示最小，所以排在最后面
for(int i=1;i<=l;i++)
{
    if(sa[i]>w)
        tp[++p]=sa[i]-w;
}
jsort(); //更新sa[]的值
```

后缀数组的核心代码，此时我们已经求出了长度为w的字符串的sa, tp, rak, 那么接下来我们需要倍增的求解长度为2w的信息。我们可以先更新tp的值(第二关键字中排名为i的位置。)

对于排名为i的字符串来说：

第一关键字的位置+w=第二关键字的位置。

后缀数组

对于此时的前 $1-w$ 个字符，他们不作为任何长度为 $2w$ 的字符串的第二关键字，那么这些位置的第二关键字都为空，相当于第二关键字最小，所以排名在最后面 $(1-w, 1)$ ，为了方便，我们也给他们从小到大排序，所以排名为 $1-w+i$ 。

对于 w 之和的位置，都会作为某个长度为 $2w$ 的第二关键字，但是要怎怎么 $O(n)$ 的计算出 tp 呢，可以利用上一次的 sa ，在长度为 $2w$ 中的第二关键字是不是就是长度为 w 的第一关键字，当他作为第一关键字时的首地址为 $sa[i]$ ，那么作为第二关键字时的首地址就再减去长度 w ，表示为 $sa[i]-w$ 。



后缀数组

更新完`tp`之和再用基数排序更新`sa`的值，那么这样就更新好了，最后再是更新`rak`的值，一定要注意，在 $w < l$ 的时候，`rak` (以`i`开头的长度为`w`的后缀的排名) 的值可能会相同的，对于`rak`相同的位置，一定要用相同的排名。

```
std::swap(tp,rak); //把tp数组当做临时的rak数组
rak[sa[1]]=p=1;
for(int i=2;i<=l;i++)
{ //这里tp实际就是rak
    if(tp[sa[i-1]]==tp[sa[i]]&&tp[sa[i-1]+w]==tp[sa[i]+w])
        //此时比较的字符串长度为2w，如果前面w的排名相同，后面w的排名也相同，那么他们仍然相同
        rak[sa[i]]=p;
    else
        rak[sa[i]]=++p; //不同位置排名相同，他们的rak值应该相同
}
```

后缀数组

什么时候，两个字符串的rak值才会相同呢？要注意sa和tp值都是不相同的，因为他表示的是位置，位置只能有一个，但是又不能遗漏，所以排名相邻的位置可能他们的字符串是一样的。所以我们要先比较rak[sa[i]]和rak[sa[i-1]]，如果相等，表示第一关键字相等，接下来再比较第二关键字，rak始终是表示以当前位置相同，所以我们在得到sa值的时候，往后面+w就表示第二关键字了，所以也就是比较rak[sa[i]+w]和rak[sa[i-1]+w]，如果两部分都想同，排名+1，否则排名不变。

sa[i]

sa[i]+w

第一关键字

第二关键字

当前字符串排名为i

后缀数组

当然，后缀数组不会这么简单的只求一个字符串的所有后缀的排名，把一个字符串的所有后缀排序之后生成的数组只是他的第一步。把后缀数组当做一个辅助工具来实现其他功能。比如求lcp(最长公共前缀)。

这里的lcp是指对已经按照顺序排好序的后缀数组进行lcp，也就是LCP(sa[i])。

先介绍lcp的一些简单性质：

$$\text{LCP}(i, j) = \text{LCP}(j, i)$$

$$\text{LCP}(i, i) = n - \text{sa}[i] + 1;$$

对于 $i \leq k \leq j$ (均表示排名)

$$\text{LCP}(i, j) = \min(\text{LCP}(i, k), \text{LCP}(k, j))$$



后缀数组

那我们就可以推导出来下一个性质；

对于 $i < k \leq j$,

$$\text{LCP}(i, j) = \min(\text{LCP}(k-1, k))$$

既然有 $\text{LCP}(i, j) = \min(\text{LCP}(i, k), \text{LCP}(k, j))$

那么就有 $\text{LCP}(i, j) = \min(\text{LCP}(i, i+1), \text{LCP}(i+1, j))$,

再把后面继续拆分, 那么就可以拆分成上面的形式

我们这里设 $\text{height}(i) = \text{LCP}(i-1, i)$, 即 $\text{height}[i]$ 代表第 i 小的后缀与第 $i-1$ 小的后缀的 LCP, 则求 $\text{LCP}(i, j)$ 就可以转换成求 $\text{height}(i+1)$ 和 $\text{height}(j)$ 之间的最小值, 如何才能快速求出呢? 很简单, RMQ 预处理就可以了, 时间复杂度为 $n \log n$, 每次查询时间复杂度为 $O(1)$.

那我们要如何求出 height 数组呢?

$$h[i] = \text{height}[\text{rak}[i]];$$

$$\text{那么 } \text{height}[i] = h[\text{sa}[i]]$$

后缀数组

```
int dp[maxn][30];
void cal_RMQ()
{
    for(int i=1;i<=l;i++)
        dp[i][0]=height[i];
    for(int j=1;(1<<j)<=l;j++)
    {
        for(int i=1;i+(1<<j)-1<=n;i++)
        {
            dp[i][j]=min(dp[i][j-1],dp[i+(1<<(j-1))][j-1]);
        }
    }
}
```

后缀数组

还需要证明最后一个公式：

$$h[i] \geq h[i-1] - 1:$$

证明：设第 $i-1$ 个字符串排名 x ，第 k 个字符串排名为 $x-1$ ，那么这里两个字符串的最长公共前缀是 $height[rak[i-1]]$ ，我们接下来讨论第 $k+1$ 个字符串和第 i 个字符串之间的关系。

(1) 如果如果 k 的首字符和 $i-1$ 的首字符不相同，那么很明显 $height[rak[i-1]] = 0$ ，那无论如何

$$height[rak[i]] \geq height[rk[i-1]] - 1, \text{ 即 } h[i] \geq h[i-1] - 1$$

(2) 如果 k 和 $i-1$ 的首字符相同，第 $k+1$ 个字符串就是第 k 个字符串去掉首字符，第 i 个字符串就是第 $i+1$ 个字符串去掉首字符，那么很明显少了一个相等的首字符，所以仍然成立，此时两者相等。

所以总的来说 $h[i] \geq h[i-1] - 1$ 。

后缀数组

那么这个公式有什么用呢？用途太大了，如果我求出来了 $h[i]$ 的答案，那么我求 $h[i+1]$ 的答案的时候，从不需要再从头开始枚举了，只需要从 $h[i]-1$ 开始往后面枚举就可以了，这样就省掉了很多时间。

```
void cal_height()
{
    for(int i=1;i<=l;i++)
        rak[sa[i]]=i;//初始化rak数组
    for(int i=1;i<=l;i++)
    {
        if(k)    k--;//公式 $h[i] \geq h[i-1]-1$ ; k就是 $h[i]$ 
        int j=sa[rak[i]-1]; //j表示排名比i小的位置
        while(s[i+k]==s[j+k])//求LCP(i-1,i)的值
            k++;
        height[rak[i]]=k;
    }
}
```

后缀数组应用

后缀数组的应用特别广泛，主要用于：

- 1: 后缀排序
- 2: 最长公共前缀
- 3: 不相同子串的个数
- 4: 最长重复子串。

洛谷 3809 后缀排序

洛谷 3181 找相同字符

洛谷 2336 喵星球上的点名

洛谷 2408 不同子串个数

P0J 2774

P0J 3693